Scoping

Using the scoping API one defines which elements are referable by a given reference. For instance, using the entities example a feature contains a cross-reference to a type:

```
datatype String

entity HasAuthor {
    author: String
}
```

The grammar rule for features looks like this:

```
Feature:
    (many?='many')? name=ID ':' type=[Type];
```

The grammar declares that for the reference *type* only instances of the type *Type* are allowed. However, this simple declaration doesn't say anything about where to find the type. That is the duty of scopes.

An IScopeProvider is responsible for providing an IScope for a given context EObject and EReference. The returned IScope should contain all target candidates for the given object and cross-reference.

```
public interface IScopeProvider {

        /**
         * Returns a scope for the given context. The scope provides access to the compatibl
e
         * visible EObjects for a given reference.
         *
         * @param context the element from which an element shall be referenced. It doesn't
need to be the element
         *                containing the reference, it is just used to find the most inner s
cope for given {@link EReference}.
         * @param reference the reference for which to get the scope.
         * @return {@link IScope} representing the innermost {@link IScope} for the
         *         passed context and reference. Note for implementors: The result may not b
e <code>null</code>.
         *         Return <code>IScope.NULLSCOPE</code> instead.
         */
    IScope getScope(EObject context, EReference reference);

}
```

Global Scopes and Resource Descriptions

In the simple scoping example above we don't have references across model files. Neither is there a concept like a namespace which would make scoping a bit more complicated. Basically, every *Element* declared in the same resource is visible by its name. However, in the real world things are most likely not that simple: What if you want to reuse certain declared elements across different files and you want to share those as library between different users? You would want to introduce some kind of cross-resource reference.

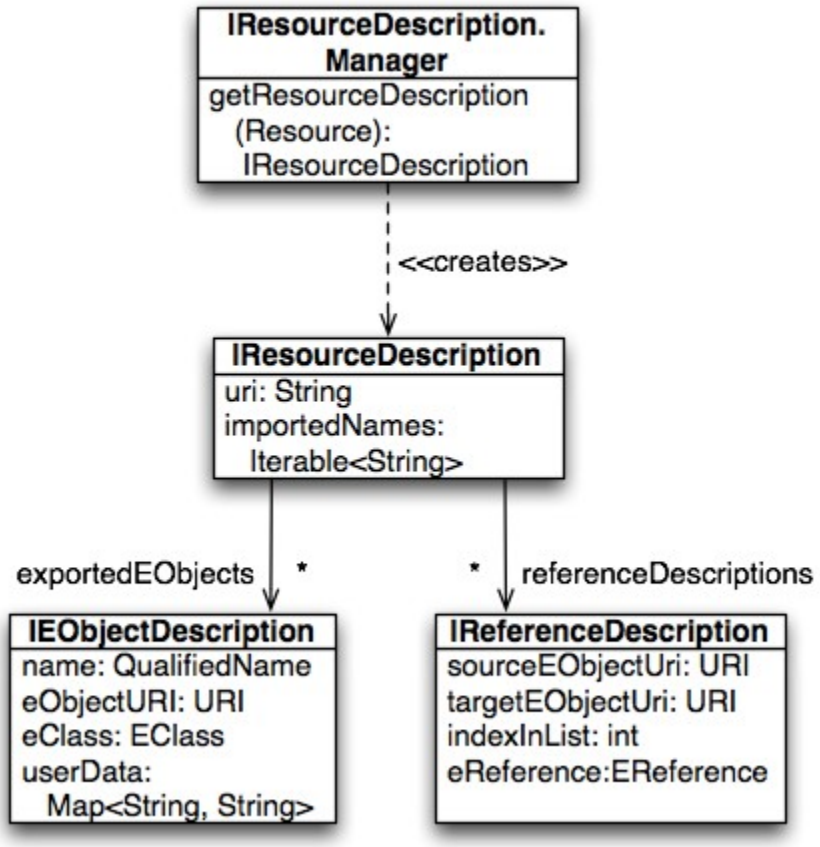
Defining what is visible from outside the current resource is the responsibility of global scopes. As the name suggests, global scopes are provided by instances of the IGlobalScopeProvider. The data structures (called index) used to store its elements are described in the next section.

Resource and EObject Descriptions

In order to make elements of one file referable from another file, you need to export them as part of a so called IResourceDescription.

An IResourceDescription contains information about the resource itself, which primarily its URI, a list of exported EObjects in the form of IEObjectDescriptions, as well as information about outgoing cross-references and qualified names it references. The cross-references contain only resolved references, while the list of imported qualified names also contains the names that couldn't be resolved. This information is leveraged by the indexing infrastructure of Xtext in order to compute the transitive hull of dependent resources.

For users, and especially in the context of scoping, the most important information is the list of exported EObjects. An IEObjectDescription stores the URI of the actual EObject, its QualifiedName, as well as its EClass. In addition one can export arbitrary information using the *user data* map. The following diagram gives an overview on the description classes and their relationships.



A language is configured with default implementations of IResourceDescription.Manager and DefaultResourceDescriptionStrategy, which are responsible to compute the list of exported IEObjectDescriptions. The Manager iterates over the whole EMF model for each Resource and asks the ResourceDescriptionStrategy to compute an IEObjectDescription for each EObject. The ResourceDescriptionStrategy applies the `getQualifiedName(EObject obj)` from IQualifiedNameProvider on the object, and if it has a qualified name an IEObjectDescription is created and passed back to the Manager which adds it to the list of exported objects. If an EObject doesn't have a qualified name, the element is considered to be not referable from outside the resource and consequently not indexed. If you don't like this behavior, you can implement and bind your own implementation of IDefaultResourceDescriptionStrategy.

There are two different default implementations of IQualifiedNameProvider. Both work by looking up an EAttribute '*name*'. The SimpleNameProvider simply returns the plain value, while the DefaultDeclarativeQualifiedNameProvider concatenates the simple name with the qualified name of its parent exported EObject. This effectively simulates the qualified name computation of most namespace-based languages such as Java. It also allows to override the name computation declaratively: Just add methods named `qualifiedName` in a subclass and give each of them one argument with the type of element you wish to compute a name for.

As already mentioned, the default implementation strategy exports every model element that the IQualifiedNameProvider can provide a name for. This is a good starting point, but when your models become bigger and you have a lot of them the index will become larger and larger. In most scenarios only a small part of your model should be visible from outside, hence only that small part needs to be in the index. In order to do this, bind a custom implementation of IDefaultResourceDescriptionStrategy and create index representations only for those elements that you want to reference from outside the resource they are contained in. From within the resource, references to those filtered elements are still possible as long as they have a name.

Beside the exported elements the index contains IReferenceDescriptions that contain the information who is referencing who. They are created through the IResourceDescription.Manager and IDefaultResourceDescriptionStrategy, too. If there is a model element that references another model element, the IDefaultResourceDescriptionStrategy creates an IReferenceDescription that contains the URI of the referencing element (*sourceEObjectURI*) and the referenced element (*targetEObjectURI*). These IReferenceDescriptions are very useful to find references and calculate affected resources.

As mentioned above, in order to compute an IResourceDescription for a resource the framework asks the IResourceDescription.Manager which delegates to the IDefaultResourceDescriptionStrategy. To convert between a QualifiedName and its String representation you can use the IQualifiedNameConverter. Here is some Xtend code showing how to do that:

```
@Inject IResourceServiceProvider.Registry rspr
@Inject IQualifiedNameConverter converter

def void printExportedObjects(Resource resource) {
    val resServiceProvider = rspr.getResourceServiceProvider(resource.URI)
    val manager = resServiceProvider.getResourceDescriptionManager()
    val description = manager.getResourceDescription(resource)
    for (eod : description.exportedObjects) {
        println(converter.toString(eod.qualifiedName))
    }
}
```

In order to obtain a Manager it is best to ask the corresponding IResourceServiceProvider as shown above. That is because each language might have a totally different implementation, and as you might refer from one language to a different language you cannot reuse the Manager of the first language.

Now that we know how to export elements to be referable from other resources, we need to learn how those exported IEObjectDescriptions can be made available to the referencing resources. That is the responsibility of global scoping which is described in the following section.

If you would like to see what's in the index, you could use the 'Open Model Element' dialog from the navigation menu entry.

Global Scopes Based On External Configuration

Instead of explicitly referring to imported resources, another option is to have some kind of external configuration in order to define what is visible from outside a resource. Java for instance uses the notion of the class path to define containers (jars and class folders) which contain referenceable elements. In the case of Java the order of such entries is also important.

To enable support for this kind of global scoping in Xtext, a DefaultGlobalScopeProvider has to be bound to the IGlobalScopeProvider interface. By default Xtext leverages the class path mechanism since it is well designed and already understood by most of our users. The available tooling provided by JDT and PDE to configure the class path adds even more value. However, it is just a default: you can reuse the infrastructure without using Java and be independent of the JDT.

In order to know what is available in the "world", a global scope provider which relies on external configuration needs to read that configuration in and be able to find all candidates for a certain EReference. If you don't want to force users to have a folder and file name structure reflecting the actual qualified names of the referenceable EObjects, you'll have to load all resources up front and either keep holding them in memory or remember all information which is needed for the resolution of cross-references. In Xtext that information is provided by a so-called IEObjectDescription.

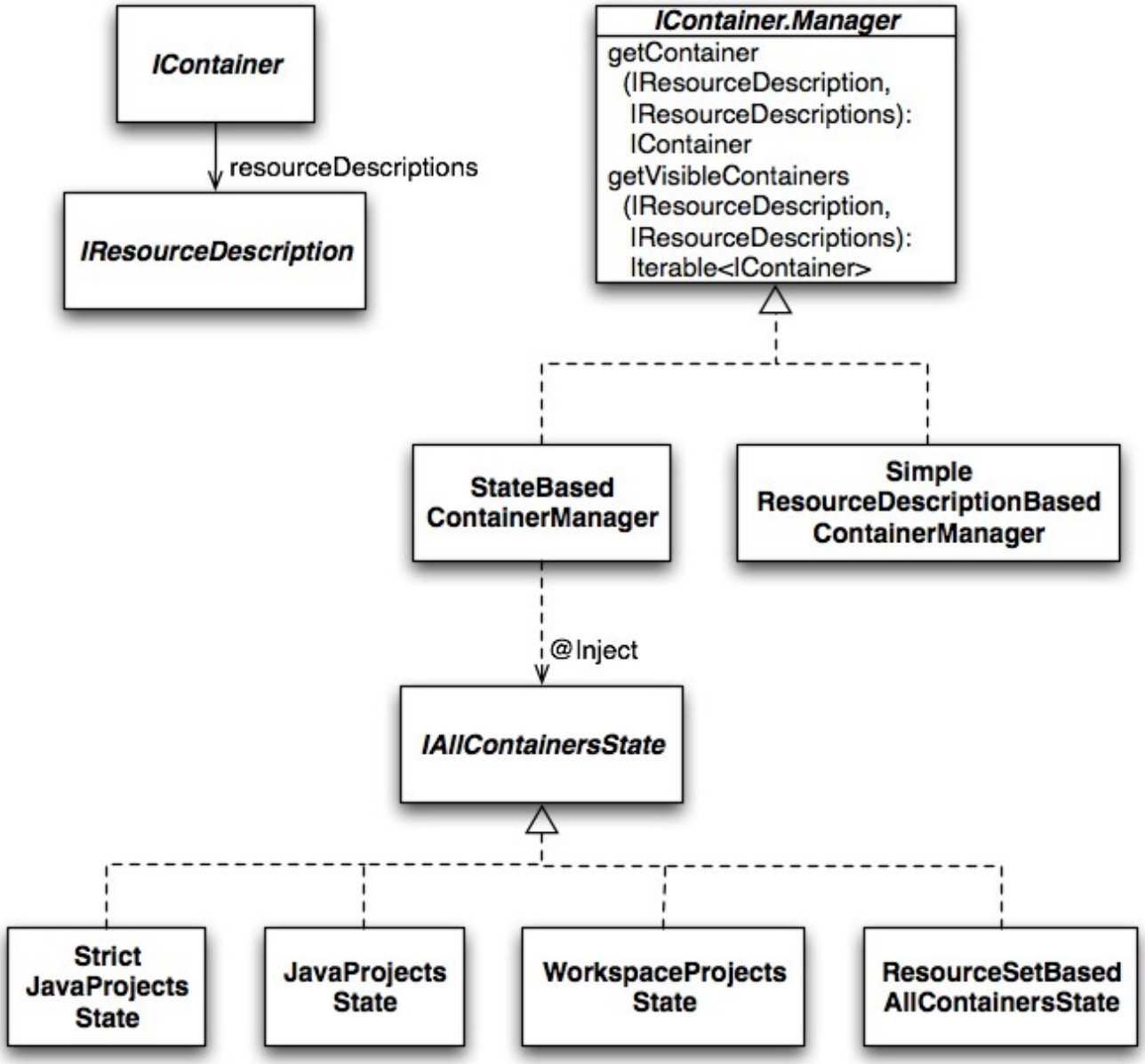
About the Index, Containers and Their Manager

Xtext ships with an index which remembers all IResourceDescription and their IEObjectDescription objects. In the IDE-context (i.e. when running the editor, etc.) the index is updated by an incremental project builder. As opposed to that, in a non-UI context you typically do not have to deal with changes, hence the infrastructure can be much simpler. In both situations the global index state is held by an implementation of IResourceDescriptions (note the plural form!). The bound singleton in the UI scenario is even aware of unsaved editor changes, such that all linking happens to the latest possibly unsaved version of the resources. You will find the Guice configuration of the global index in the UI scenario in SharedModule.

The index is basically a flat list of instances of IResourceDescription. The index itself doesn't know about visibility constraints due to class path restriction. Rather than that, they are defined by the referencing language by means of so called IContainers: While Java might load a resource via ClassLoader.loadResource() (i.e. using the class path mechanism), another language could load the same resource using the file system paths.

Consequently, the information which container a resource belongs to depends on the referencing context. Therefore an IResourceServiceProvider provides another interesting service, which is called IContainer.Manager. For a given IResourceDescription, the Manager provides you the IContainer as well as a list of all IContainers which are visible from there. Note that the index is globally shared between all languages while the Manager, which adds the semantics of containers, can be very different depending on the language. The following method lists all resources visible from a given Resource:

```
@Inject IContainer.Manager manager
```



in the Eclipse UI module of the referencing language. The latter looks a bit more difficult than a common binding, as we have to bind a global singleton to a Guice provider. A StrictJavaProjectsState requires all elements to be on the class path, while the default JavaProjectsState also allows models in non-source folders.

Eclipse Project-Based Containers

If the class path based mechanism doesn't work for your case, Xtext offers an alternative container manager based on plain Eclipse projects: Each project acts as a container and the project references (*Properties → Project References*) are the visible containers.

In this case, your runtime module should use the StateBasedContainerManager as shown above and the Eclipse UI module should bind

```
1. public Provider<IAllContainersState> provideIAllContainersState() {
2.     return org.eclipse.xtext.ui.shared.Access.getWorkspaceProjectsState();
3. }
```

ResourceSet-Based Containers

If you need an IContainer.Manager that is independent of Eclipse projects, you can use the ResourceSetBasedAllContainersState. This one can be configured with a mapping of container handles to resource URIs.

```
1. public Provider<IAllContainersState> provideIAllContainersState() {
2.     return org.eclipse.xtext.ui.shared.Access.getJavaProjectsState();
3. }
```

local scoping

Local Scoping

We now know how the outer world of referenceable elements can be defined in Xtext. Nevertheless, not everything is available in all contexts and with a global name. Rather than that, each context can usually have a different scope. As already stated, scopes can be nested, i.e. a scope can contain elements of a parent scope in addition to its own elements. When parent and child scope contain different elements with the same name, the parent scope's element will usually be *shadowed* by the element from the child scope.

To illustrate that, let's have a look at Java: Java defines multiple kinds of scopes (object scope, type scope, etc.). For Java one would create the scope hierarchy as commented in the following example:

```
1. // file contents scope
2. import static my.Constants.STATIC;
3.
4. public class ScopeExample { // class body scope
5.   private Object field = STATIC;
6.
7.   private void method(String param) { // method body scope
8.     String localVar = "bar";
9.     innerBlock: { // block scope
10.      String innerScopeVar = "foo";
11.      Object field = innerScopeVar;
12.      // the scope hierarchy at this point would look like this:
13.      //  blockScope{field,innerScopeVar}->
14.      //  methodScope{localVar, param}->
15.      //  classScope{field}-> ('field' is shadowed)
16.      //  fileScope{STATIC}->
17.      //  classpathScope{
18.      //      'all qualified names of accessible static fields'} ->
19.      //  NULLSCOPE{}
20.      //
21.    }
22.    field.add(localVar);
23.  }
24. }
```

In fact the class path scope should also reflect the order of class path entries. For instance:

```
1. classpathScope{stuff from bin/}
2. -> classpathScope{stuff from foo.jar/}
3. -> ...
4. -> classpathScope{stuff from JRE System Library}
5. -> NULLSCOPE{}
```

Please find the motivation behind this and some additional details in this blog post.

Imported Namespace Aware Scoping

The imported namespace aware scoping is based on qualified names and namespaces. It adds namespace support to your language, which is comparable and similar to namespaces in Scala and C#. Scala and C# both allow to have multiple nested packages within one file, and you can put imports per namespace, such that imported names are only visible within that namespace. See the domain model example: its scope provider extends ImportedNamespaceAwareLocalScopeProvider.

Importing Namespaces

The ImportedNamespaceAwareLocalScopeProvider looks up EAttributes with name *importedNamespace* and interprets them as import statements.

```
PackageDeclaration:
    'package' name=QualifiedName '{'
        (elements+=AbstractElement)*
    '}';

AbstractElement:
    PackageDeclaration | Type | Import;

QualifiedName:
    ID ('.' ID)*;

Import:
    'import' importedNamespace=QualifiedNameWithWildcard;

QualifiedNameWithWildcard:
    QualifiedName '.*'?;
```

Qualified names with or without a wildcard at the end are supported. For an import of a qualified name the simple name is made available as we know from e.g. Java, where `import java.util.Set` makes it possible to refer to java.util.Set by its simple name *Set*. Contrary to Java, the import is not active for the whole file, but only for the namespace it is declared in and its child namespaces. That is why you can write the following in the example DSL:

```
package foo {
    import bar.Foo
    entity Bar extends Foo {
    }
}

package bar {
    entity Foo {}
}
```

Of course the declared elements within a package are as well referable by their simple name:

```
package bar {
    entity Bar extends Foo {}
    entity Foo {}
}
```